



Pandas : Data Treatment Library

Ou comment gérer de la donnée de manière standardisée



Pandas

- Wes McKinney
- Open source
- 1st release : 11 janvier 2008
- Written in C / Cython



Pandas

```
pip install pandas
```

```
import pandas as pd  
import numpy as np
```

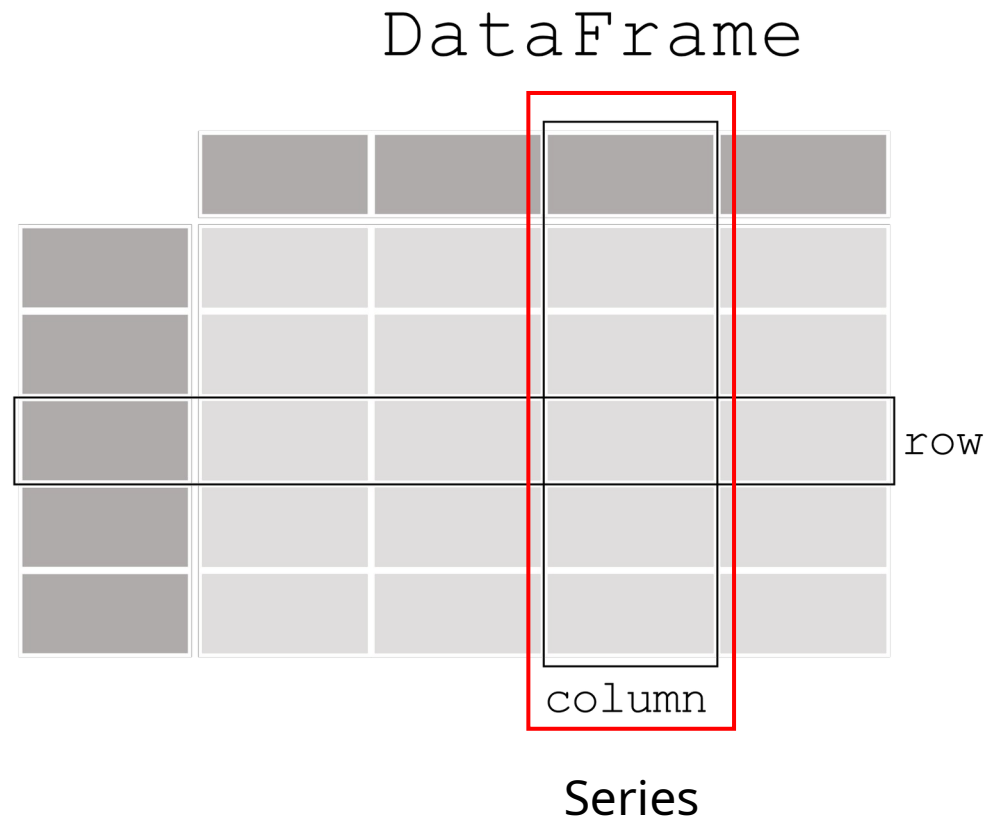


Pandas : Concepts

- DataFrame (the basic element)
- Series



Pandas : Series



Pandas : Series

```
In [3]: s = pd.Series(np.random.randn(5), index=["a", "b", "c", "d", "e"])

In [4]: s
Out[4]:
a    0.469112
b   -0.282863
c   -1.509059
d   -1.135632
e    1.212112
dtype: float64

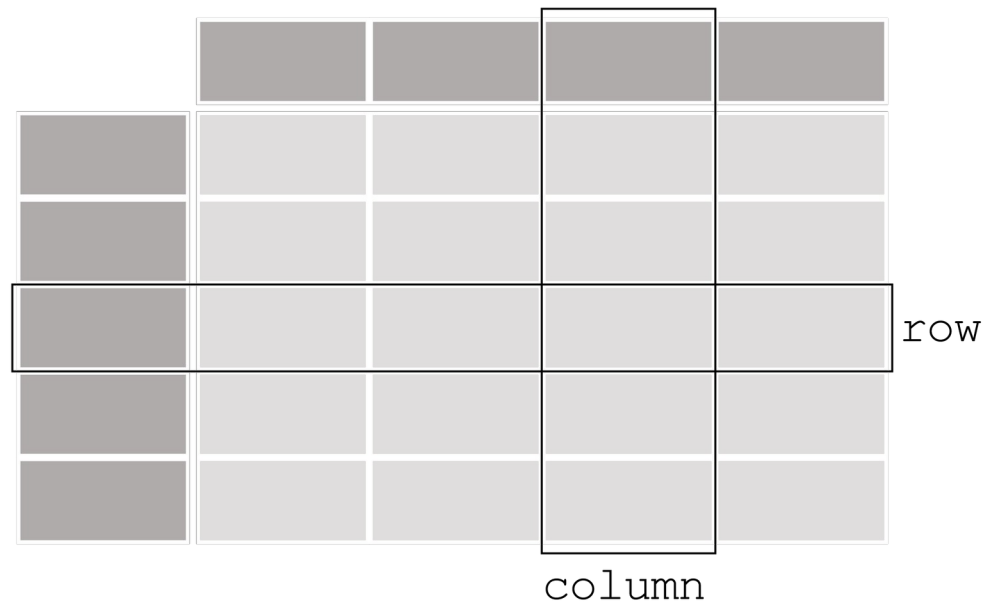
In [5]: s.index
Out[5]: Index(['a', 'b', 'c', 'd', 'e'], dtype='object')

In [6]: pd.Series(np.random.randn(5))
Out[6]:
0   -0.173215
1    0.119209
2   -1.044236
3   -0.861849
4   -2.104569
dtype: float64
```



Pandas : DataFrames

DataFrame



Pandas : DataFrames

```
In [37]: d = {  
.....:     "one": pd.Series([1.0, 2.0, 3.0], index=["a", "b", "c"]),  
.....:     "two": pd.Series([1.0, 2.0, 3.0, 4.0], index=["a", "b", "c", "d"]),  
.....: }
```

```
In [38]: df = pd.DataFrame(d)
```

```
In [39]: df
```

```
Out[39]:
```

	one	two
a	1.0	1.0
b	2.0	2.0
c	3.0	3.0
d	NaN	4.0

```
In [40]: pd.DataFrame(d, index=["d", "b", "a"])
```

```
Out[40]:
```

	one	two
d	NaN	4.0
b	2.0	2.0
a	1.0	1.0

```
In [41]: pd.DataFrame(d, index=["d", "b", "a"], columns=["two", "three"])
```

```
Out[41]:
```

	two	three
d	4.0	NaN
b	2.0	NaN
a	1.0	NaN

Pandas : Index / Columns

```
In [42]: df.index  
Out[42]: Index(['a', 'b', 'c', 'd'], dtype='object')  
  
In [43]: df.columns  
Out[43]: Index(['one', 'two'], dtype='object')
```



Pandas : DataFrames I/O

```
>>> pd.read_csv('data.csv')
```

```
>>> df = pd.DataFrame({'name': ['Raphael', 'Donatello'],  
...                    'mask': ['red', 'purple'],  
...                    'weapon': ['sai', 'bo staff']})  
>>> df.to_csv(index=False)  
'name,mask,weapon\nRaphael,red,sai\nDonatello,purple,bo staff\n'
```

```
>>> compression_opts = dict(method='zip',  
...                          archive_name='out.csv')  
>>> df.to_csv('out.zip', index=False,  
...          compression=compression_opts)
```

Pandas : DataFrames I/O

Multiple type of I/O support :

- CSV
- EXCEL
- LATEX
- HTML
- JSON
- HDFS
- SQL
- ...



Indexing / Selecting data



Pandas : Indexing / Selecting data

```
In [1]: dates = pd.date_range('1/1/2000', periods=8)

In [2]: df = pd.DataFrame(np.random.randn(8, 4),
...:                       index=dates, columns=['A', 'B', 'C', 'D'])
...:

In [3]: df
Out[3]:
```

	A	B	C	D
2000-01-01	0.469112	-0.282863	-1.509059	-1.135632
2000-01-02	1.212112	-0.173215	0.119209	-1.044236
2000-01-03	-0.861849	-2.104569	-0.494929	1.071804
2000-01-04	0.721555	-0.706771	-1.039575	0.271860
2000-01-05	-0.424972	0.567020	0.276232	-1.087401
2000-01-06	-0.673690	0.113648	-1.478427	0.524988
2000-01-07	0.404705	0.577046	-1.715002	-1.039268
2000-01-08	-0.370647	-1.157892	-1.344312	0.844885

Pandas : Indexing / Selecting data

Thus, as per above, we have the most basic indexing using []:

```
In [4]: s = df['A']  
  
In [5]: s[dates[5]]  
Out[5]: -0.6736897080883706
```



Pandas : Indexing / Selecting data

You can pass a list of columns to `[]` to select columns in that order. If a column is not contained in the DataFrame, an exception will be raised. Multiple columns can also be set in this manner:

```
In [6]: df
Out[6]:
```

	A	B	C	D
2000-01-01	0.469112	-0.282863	-1.509059	-1.135632
2000-01-02	1.212112	-0.173215	0.119209	-1.044236
2000-01-03	-0.861849	-2.104569	-0.494929	1.071804
2000-01-04	0.721555	-0.706771	-1.039575	0.271860
2000-01-05	-0.424972	0.567020	0.276232	-1.087401
2000-01-06	-0.673690	0.113648	-1.478427	0.524988
2000-01-07	0.404705	0.577046	-1.715002	-1.039268
2000-01-08	-0.370647	-1.157892	-1.344312	0.844885

```
In [7]: df[['B', 'A']] = df[['A', 'B']]
```

```
In [8]: df
Out[8]:
```

	A	B	C	D
2000-01-01	-0.282863	0.469112	-1.509059	-1.135632
2000-01-02	-0.173215	1.212112	0.119209	-1.044236
2000-01-03	-2.104569	-0.861849	-0.494929	1.071804
2000-01-04	-0.706771	0.721555	-1.039575	0.271860
2000-01-05	0.567020	-0.424972	0.276232	-1.087401
2000-01-06	0.113648	-0.673690	-1.478427	0.524988
2000-01-07	0.577046	0.404705	-1.715002	-1.039268
2000-01-08	-1.157892	-0.370647	-1.344312	0.844885

Pandas : Indexing / Selecting data

With Series, the syntax works exactly as with an ndarray, returning a slice of the values and the corresponding labels:

```
In [27]: s[:5]
Out[27]:
2000-01-01    0.469112
2000-01-02    1.212112
2000-01-03   -0.861849
2000-01-04    0.721555
2000-01-05   -0.424972
Freq: D, Name: A, dtype: float64

In [28]: s[::2]
Out[28]:
2000-01-01    0.469112
2000-01-03   -0.861849
2000-01-05   -0.424972
2000-01-07    0.404705
Freq: 2D, Name: A, dtype: float64

In [29]: s[::-1]
Out[29]:
2000-01-08   -0.370647
2000-01-07    0.404705
2000-01-06   -0.673690
2000-01-05   -0.424972
2000-01-04    0.721555
2000-01-03   -0.861849
2000-01-02    1.212112
2000-01-01    0.469112
Freq: -1D, Name: A, dtype: float64
```


Pandas : Label selection

```
In [38]: s1 = pd.Series(np.random.randn(6), index=list('abcdef'))
```

```
In [39]: s1
```

```
Out[39]:
```

```
a    1.431256  
b    1.340309  
c   -1.170299  
d   -0.226169  
e    0.410835  
f    0.813850  
dtype: float64
```

```
In [40]: s1.loc['c:']
```

```
Out[40]:
```

```
c   -1.170299  
d   -0.226169  
e    0.410835  
f    0.813850  
dtype: float64
```

```
In [41]: s1.loc['b']
```

```
Out[41]: 1.3403088497993827
```



Pandas : Label selection

With a DataFrame:

```
In [44]: df1 = pd.DataFrame(np.random.randn(6, 4),  
.....:                    index=list('abcdef'),  
.....:                    columns=list('ABCD'))  
.....:
```

```
In [45]: df1
```

```
Out[45]:
```

	A	B	C	D
a	0.132003	-0.827317	-0.076467	-1.187678
b	1.130127	-1.436737	-1.413681	1.607920
c	1.024180	0.569605	0.875906	-2.211372
d	0.974466	-2.006747	-0.410001	-0.078638
e	0.545952	-1.219217	-1.226825	0.769804
f	-1.281247	-0.727707	-0.121306	-0.097883

```
In [46]: df1.loc[['a', 'b', 'd'], :]
```

```
Out[46]:
```

	A	B	C	D
a	0.132003	-0.827317	-0.076467	-1.187678
b	1.130127	-1.436737	-1.413681	1.607920
d	0.974466	-2.006747	-0.410001	-0.078638

Pandas : Position selection

```
In [61]: s1 = pd.Series(np.random.randn(5), index=list(range(0, 10, 2)))
```

```
In [62]: s1
```

```
Out[62]:
```

```
0    0.695775  
2    0.341734  
4    0.959726  
6   -1.110336  
8   -0.619976  
dtype: float64
```

```
In [63]: s1.iloc[:3]
```

```
Out[63]:
```

```
0    0.695775  
2    0.341734  
4    0.959726  
dtype: float64
```

```
In [64]: s1.iloc[3]
```

```
Out[64]: -1.110336102891167
```



Pandas : Position selection

Select via integer slicing:

```
In [69]: df1.iloc[:3]
Out[69]:
```

	0	2	4	6
0	0.149748	-0.732339	0.687738	0.176444
2	0.403310	-0.154951	0.301624	-2.179861
4	-1.369849	-0.954208	1.462696	-1.743161

```
In [70]: df1.iloc[1:5, 2:4]
Out[70]:
```

	4	6
2	0.301624	-2.179861
4	1.462696	-1.743161
6	1.314232	0.690579
8	0.014871	3.357427

Pandas : Boolean mask selection

You may select rows from a DataFrame using a boolean vector the same length as the DataFrame's index (for example, something derived from one of the columns of the DataFrame):

```
In [153]: df[df['A'] > 0]
```

```
Out[153]:
```

	A	B	C	D	E	Ø
2000-01-01	0.469112	-0.282863	-1.509059	-1.135632	NaN	NaN
2000-01-02	1.212112	-0.173215	0.119209	-1.044236	NaN	NaN
2000-01-04	7.000000	-0.706771	-1.039575	0.271860	NaN	NaN
2000-01-07	0.404705	0.577046	-1.715002	-1.039268	NaN	NaN

Pandas : Set / Reset index

```
In [339]: data
Out[339]:
   a    b  c  d
0 bar one z  1.0
1 bar two y  2.0
2 foo one x  3.0
3 foo two w  4.0

In [340]: indexed1 = data.set_index('c')

In [341]: indexed1
Out[341]:
   a    b  d
c
z bar one  1.0
y bar two  2.0
x foo one  3.0
w foo two  4.0

In [342]: indexed2 = data.set_index(['a', 'b'])

In [343]: indexed2
Out[343]:
   c  d
a  b
bar one z  1.0
     two y  2.0
foo one x  3.0
     two w  4.0
```

Pandas : Set / Reset index

```
In [350]: data
Out[350]:
```

			c	d
a	b			
bar	one	z	1.0	
	two	y	2.0	
foo	one	x	3.0	
	two	w	4.0	

```
In [351]: data.reset_index()
Out[351]:
```

	a	b	c	d
0	bar	one	z	1.0
1	bar	two	y	2.0
2	foo	one	x	3.0
3	foo	two	w	4.0



Combinations



Pandas : Concatenation

```
In [1]: df1 = pd.DataFrame(  
...:     {  
...:         "A": ["A0", "A1", "A2", "A3"],  
...:         "B": ["B0", "B1", "B2", "B3"],  
...:         "C": ["C0", "C1", "C2", "C3"],  
...:         "D": ["D0", "D1", "D2", "D3"],  
...:     },  
...:     index=[0, 1, 2, 3],  
...: )  
...:  
  
In [2]: df2 = pd.DataFrame(  
...:     {  
...:         "A": ["A4", "A5", "A6", "A7"],  
...:         "B": ["B4", "B5", "B6", "B7"],  
...:         "C": ["C4", "C5", "C6", "C7"],  
...:         "D": ["D4", "D5", "D6", "D7"],  
...:     },  
...:     index=[4, 5, 6, 7],  
...: )  
...:  
  
In [3]: df3 = pd.DataFrame(  
...:     {  
...:         "A": ["A8", "A9", "A10", "A11"],  
...:         "B": ["B8", "B9", "B10", "B11"],  
...:         "C": ["C8", "C9", "C10", "C11"],  
...:         "D": ["D8", "D9", "D10", "D11"],  
...:     },  
...:     index=[8, 9, 10, 11],  
...: )  
...:
```

```
In [4]: frames = [df1, df2, df3]
```

```
In [5]: result = pd.concat(frames)
```

Pandas : Concatenation

df1					Result				
	A	B	C	D		A	B	C	D
0	A0	B0	C0	D0	0	A0	B0	C0	D0
1	A1	B1	C1	D1	1	A1	B1	C1	D1
2	A2	B2	C2	D2	2	A2	B2	C2	D2
3	A3	B3	C3	D3	3	A3	B3	C3	D3
df2					4	A4	B4	C4	D4
	A	B	C	D	5	A5	B5	C5	D5
4	A4	B4	C4	D4	6	A6	B6	C6	D6
5	A5	B5	C5	D5	7	A7	B7	C7	D7
6	A6	B6	C6	D6	8	A8	B8	C8	D8
7	A7	B7	C7	D7	9	A9	B9	C9	D9
df3					10	A10	B10	C10	D10
	A	B	C	D	11	A11	B11	C11	D11
8	A8	B8	C8	D8					
9	A9	B9	C9	D9					
10	A10	B10	C10	D10					
11	A11	B11	C11	D11					



Pandas : Concatenation

```
In [8]: df4 = pd.DataFrame(  
...:     {  
...:         "B": ["B2", "B3", "B6", "B7"],  
...:         "D": ["D2", "D3", "D6", "D7"],  
...:         "F": ["F2", "F3", "F6", "F7"],  
...:     },  
...:     index=[2, 3, 6, 7],  
...: )  
...:  
  
In [9]: result = pd.concat([df1, df4], axis=1)
```



Pandas : Concatenation

df1					df4				Result							
	A	B	C	D		B	D	F		A	B	C	D	B	D	F
0	A0	B0	C0	D0	2	B2	D2	F2	0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	C1	D1	3	B3	D3	F3	1	A1	B1	C1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	6	B6	D6	F6	2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	7	B7	D7	F7	3	A3	B3	C3	D3	B3	D3	F3
									6	NaN	NaN	NaN	NaN	B6	D6	F6
									7	NaN	NaN	NaN	NaN	B7	D7	F7



Pandas : Concatenation

A useful shortcut to `concat()` are the `append()` instance methods on `Series` and `DataFrame`. These methods actually predated `concat`. They concatenate along `axis=0`, namely the index:

```
In [13]: result = df1.append(df2)
```



Pandas : Concatenation

df1					Result				
	A	B	C	D		A	B	C	D
0	A0	B0	C0	D0	0	A0	B0	C0	D0
1	A1	B1	C1	D1	1	A1	B1	C1	D1
2	A2	B2	C2	D2	2	A2	B2	C2	D2
3	A3	B3	C3	D3	3	A3	B3	C3	D3
df2					4	A4	B4	C4	D4
	A	B	C	D	5	A5	B5	C5	D5
4	A4	B4	C4	D4	6	A6	B6	C6	D6
5	A5	B5	C5	D5	7	A7	B7	C7	D7
6	A6	B6	C6	D6					
7	A7	B7	C7	D7					



Pandas : SQL-like merging

```
In [39]: left = pd.DataFrame(  
.....:     {  
.....:         "key": ["K0", "K1", "K2", "K3"],  
.....:         "A": ["A0", "A1", "A2", "A3"],  
.....:         "B": ["B0", "B1", "B2", "B3"],  
.....:     }  
.....: )  
.....:  
  
In [40]: right = pd.DataFrame(  
.....:     {  
.....:         "key": ["K0", "K1", "K2", "K3"],  
.....:         "C": ["C0", "C1", "C2", "C3"],  
.....:         "D": ["D0", "D1", "D2", "D3"],  
.....:     }  
.....: )  
.....:  
  
In [41]: result = pd.merge(left, right, on="key")
```



Pandas : SQL-like merging

left				right				Result					
	key	A	B		key	C	D		key	A	B	C	D
0	K0	A0	B0	0	K0	C0	D0	0	K0	A0	B0	C0	D0
1	K1	A1	B1	1	K1	C1	D1	1	K1	A1	B1	C1	D1
2	K2	A2	B2	2	K2	C2	D2	2	K2	A2	B2	C2	D2
3	K3	A3	B3	3	K3	C3	D3	3	K3	A3	B3	C3	D3



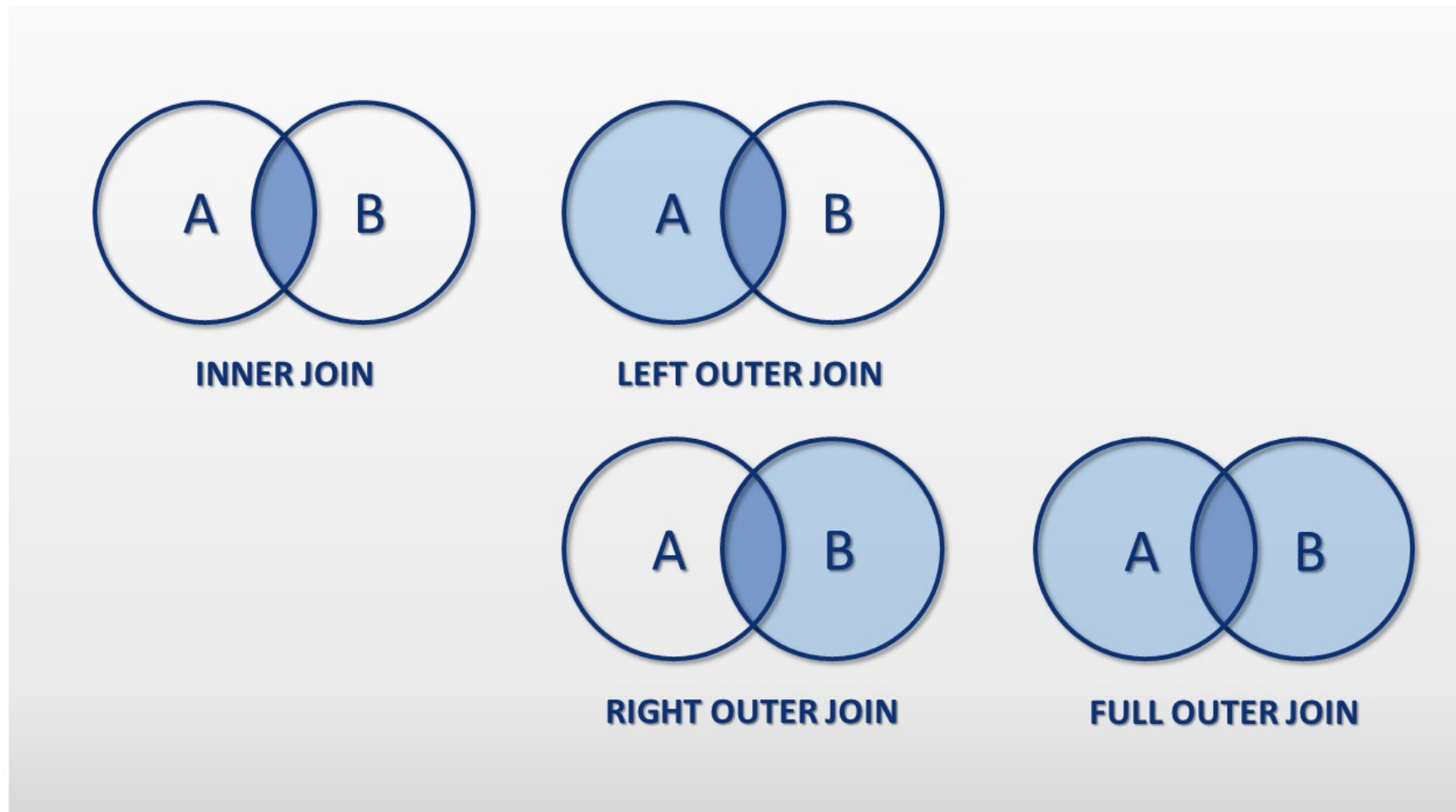
Pandas : SQL-like merging

Merge method	SQL Join Name	Description
left	LEFT OUTER JOIN	Use keys from left frame only
right	RIGHT OUTER JOIN	Use keys from right frame only
outer	FULL OUTER JOIN	Use union of keys from both frames
inner	INNER JOIN	Use intersection of keys from both frames

```
In [45]: result = pd.merge(left, right, how="left", on=["key1", "key2"])
```

```
In [47]: result = pd.merge(left, right, how="outer", on=["key1", "key2"])
```

Pandas : SQL-like merging



Missing Data



Pandas : Detecting missing data

```
In [6]: df2
Out[6]:
```

	one	two	three	four	five
a	0.469112	-0.282863	-1.509059	bar	True
b	NaN	NaN	NaN	NaN	NaN
c	-1.135632	1.212112	-0.173215	bar	False
d	NaN	NaN	NaN	NaN	NaN
e	0.119209	-1.044236	-0.861849	bar	True
f	-2.104569	-0.494929	1.071804	bar	False
g	NaN	NaN	NaN	NaN	NaN
h	0.721555	-0.706771	-1.039575	bar	True



Pandas : Detecting missing data

```
In [7]: df2["one"]  
Out[7]:  
a      0.469112  
b           NaN  
c     -1.135632  
d           NaN  
e      0.119209  
f     -2.104569  
g           NaN  
h      0.721555  
Name: one, dtype: float64
```

```
In [8]: pd.isna(df2["one"])  
Out[8]:  
a      False  
b       True  
c      False  
d       True  
e      False  
f      False  
g       True  
h      False  
Name: one, dtype: bool
```



Pandas : Detecting missing data

```
In [7]: df2["one"]  
Out[7]:  
a      0.469112  
b           NaN  
c     -1.135632  
d           NaN  
e      0.119209  
f     -2.104569  
g           NaN  
h      0.721555  
Name: one, dtype: float64
```

```
In [9]: df2["four"].notna()  
Out[9]:  
a      True  
b     False  
c      True  
d     False  
e      True  
f      True  
g     False  
h      True  
Name: four, dtype: bool
```



Pandas : Detecting missing data

```
In [10]: df2.isna()
Out[10]:
```

	one	two	three	four	five
a	False	False	False	False	False
b	True	True	True	True	True
c	False	False	False	False	False
d	True	True	True	True	True
e	False	False	False	False	False
f	False	False	False	False	False
g	True	True	True	True	True
h	False	False	False	False	False



Pandas : Datetime missing data

```
In [16]: df2["timestamp"] = pd.Timestamp("20120101")
```

```
In [17]: df2
```

```
Out[17]:
```

	one	two	three	four	five	timestamp
a	0.469112	-0.282863	-1.509059	bar	True	2012-01-01
c	-1.135632	1.212112	-0.173215	bar	False	2012-01-01
e	0.119209	-1.044236	-0.861849	bar	True	2012-01-01
f	-2.104569	-0.494929	1.071804	bar	False	2012-01-01
h	0.721555	-0.706771	-1.039575	bar	True	2012-01-01

```
In [18]: df2.loc[["a", "c", "h"], ["one", "timestamp"]] = np.nan
```

```
In [19]: df2
```

```
Out[19]:
```

	one	two	three	four	five	timestamp
a	NaN	-0.282863	-1.509059	bar	True	NaT
c	NaN	1.212112	-0.173215	bar	False	NaT
e	0.119209	-1.044236	-0.861849	bar	True	2012-01-01
f	-2.104569	-0.494929	1.071804	bar	False	2012-01-01
h	NaN	-0.706771	-1.039575	bar	True	NaT

Pandas : Fill missing data

```
In [43]: df2.fillna(0)
```

```
Out[43]:
```

	one	two	three	four	five	timestamp
a	0.000000	-0.282863	-1.509059	bar	True	0
c	0.000000	1.212112	-0.173215	bar	False	0
e	0.119209	-1.044236	-0.861849	bar	True	2012-01-01 00:00:00
f	-2.104569	-0.494929	1.071804	bar	False	2012-01-01 00:00:00
h	0.000000	-0.706771	-1.039575	bar	True	0



Pandas : Fill missing data

Method	Action
pad / ffill	Fill values forward
bfill / backfill	Fill values backward



Pandas : Fill missing data

```
In [45]: df
Out[45]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	0.119209	-1.044236	-0.861849
f	-2.104569	-0.494929	1.071804
h	NaN	-0.706771	-1.039575

```
In [46]: df.fillna(method="pad")
Out[46]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	0.119209	-1.044236	-0.861849
f	-2.104569	-0.494929	1.071804
h	-2.104569	-0.706771	-1.039575



Pandas : Drop missing data

```
In [57]: df
```

```
Out[57]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	NaN	0.000000	0.000000
f	NaN	0.000000	0.000000
h	NaN	-0.706771	-1.039575

```
In [59]: df.dropna(axis=1)
```

```
Out[59]:
```

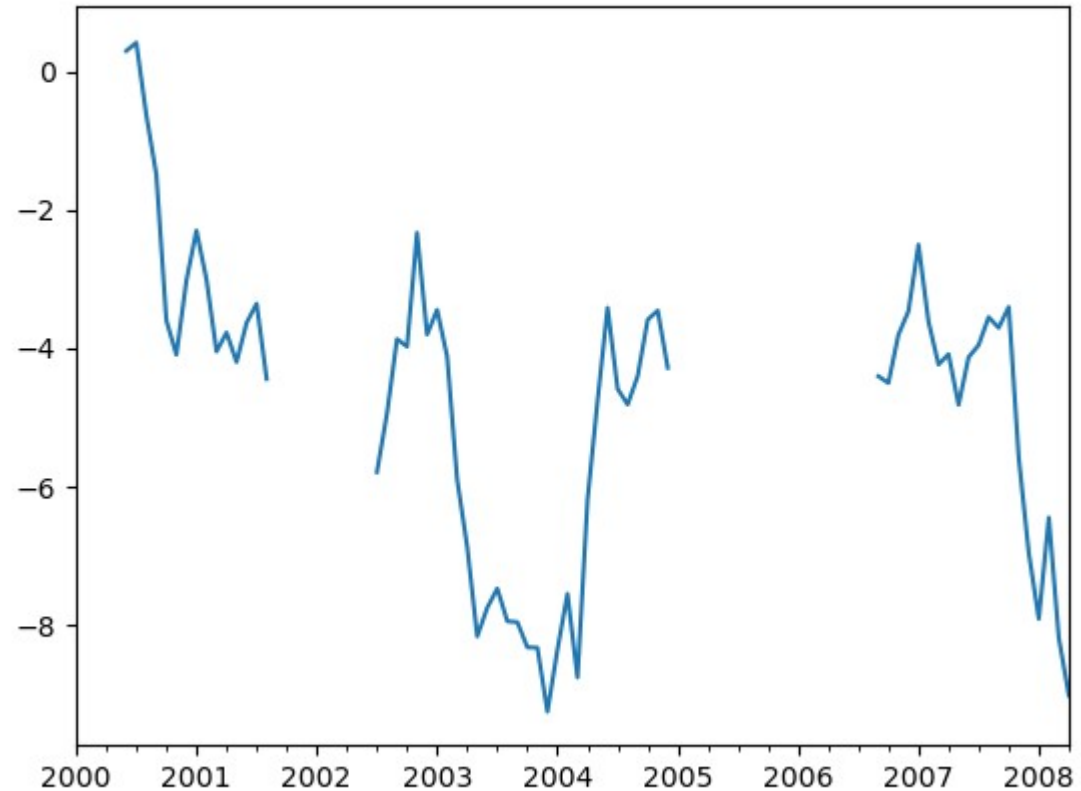
	two	three
a	-0.282863	-1.509059
c	1.212112	-0.173215
e	0.000000	0.000000
f	0.000000	0.000000
h	-0.706771	-1.039575



Pandas : Advanced filling missing data

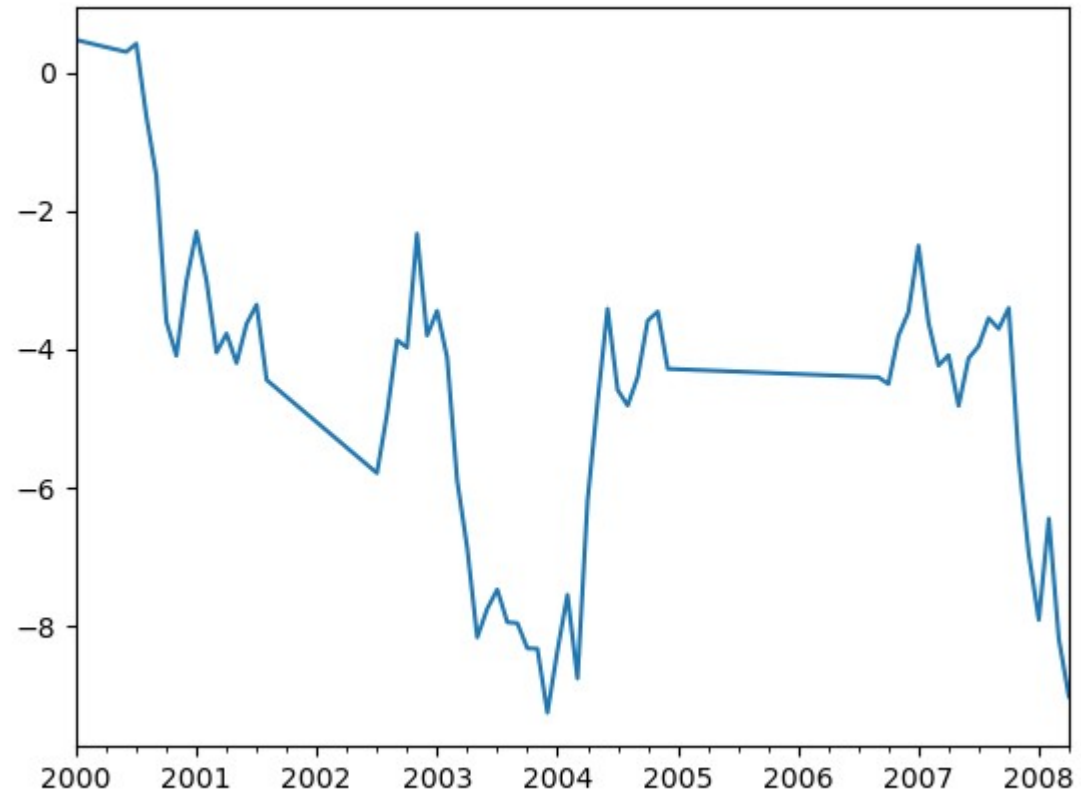
```
In [61]: ts
Out[61]:
2000-01-31    0.469112
2000-02-29         NaN
2000-03-31         NaN
2000-04-28         NaN
2000-05-31         NaN
...
2007-12-31   -6.950267
2008-01-31   -7.904475
2008-02-29   -6.441779
2008-03-31   -8.184940
2008-04-30   -9.011531
Freq: BM, Length: 100, dtype: float64
```

```
In [63]: ts.plot()
Out[63]: <AxesSubplot:>
```



Pandas : Advanced filling missing data

```
In [64]: ts.interpolate()  
Out[64]:  
2000-01-31    0.469112  
2000-02-29    0.434469  
2000-03-31    0.399826  
2000-04-28    0.365184  
2000-05-31    0.330541  
...  
2007-12-31   -6.950267  
2008-01-31   -7.904475  
2008-02-29   -6.441779  
2008-03-31   -8.184940  
2008-04-30   -9.011531  
Freq: BM, Length: 100, dtype: float64
```



Pandas : Advanced filling missing data

Multiple methods available :

- Linear
- Time
- Pad
- Slinear
- Quadratic
- Cubic
- Spline
- Barycentric
- ...



Data Operations



Pandas : Duplicates management

```
In [13]: df2
Out[13]:
   A
a  0
a  1
b  2

In [14]: df2.index.is_unique
Out[14]: False

In [15]: df2.columns.is_unique
Out[15]: True
```



Pandas : Duplicates management

```
>>> df = pd.DataFrame({
...     'brand': ['Yum Yum', 'Yum Yum', 'Indomie', 'Indomie', 'Indomie'],
...     'style': ['cup', 'cup', 'cup', 'pack', 'pack'],
...     'rating': [4, 4, 3.5, 15, 5]
... })
>>> df
   brand style  rating
0  Yum Yum   cup    4.0
1  Yum Yum   cup    4.0
2  Indomie   cup    3.5
3  Indomie  pack   15.0
4  Indomie  pack    5.0
```

```
>>> df.drop_duplicates()
   brand style  rating
2  Indomie   cup    3.5
3  Indomie  pack   15.0
4  Indomie  pack    5.0
```



Pandas : Duplicates management

```
>>> df = pd.DataFrame({
...     'brand': ['Yum Yum', 'Yum Yum', 'Indomie', 'Indomie', 'Indomie'],
...     'style': ['cup', 'cup', 'cup', 'pack', 'pack'],
...     'rating': [4, 4, 3.5, 15, 5]
... })
>>> df
   brand style  rating
0  Yum Yum   cup    4.0
1  Yum Yum   cup    4.0
2  Indomie   cup    3.5
3  Indomie  pack   15.0
4  Indomie  pack    5.0
```

```
>>> df.drop_duplicates(subset=['brand'])
   brand style  rating
0  Yum Yum   cup    4.0
2  Indomie   cup    3.5
```



Pandas : Shape

```
>>> df = pd.DataFrame({'col1': [1, 2], 'col2': [3, 4]})  
>>> df.shape  
(2, 2)
```

```
>>> df = pd.DataFrame({'col1': [1, 2], 'col2': [3, 4],  
...                   'col3': [5, 6]})  
>>> df.shape  
(2, 3)
```



Pandas : Empty check

```
>>> df_empty = pd.DataFrame({'A' : []})  
>>> df_empty  
Empty DataFrame  
Columns: [A]  
Index: []  
>>> df_empty.empty  
True
```



Pandas : Copy

```
>>> s = pd.Series([1, 2], index=["a", "b"])
>>> deep = s.copy()
>>> shallow = s.copy(deep=False)
```



Pandas : Iteration

```
>>> df = pd.DataFrame([[1, 1.5]], columns=['int', 'float'])
>>> row = next(df.iterrows())[1]
>>> row
int      1.0
float    1.5
Name: 0, dtype: float64
>>> print(row['int'].dtype)
float64
>>> print(df['int'].dtype)
int64
```



Pandas : Iteration

```
>>> df = pd.DataFrame({'num_legs': [4, 2], 'num_wings': [0, 2]},  
...                     index=['dog', 'hawk'])  
>>> df  
      num_legs  num_wings  
dog           4          0  
hawk          2          2  
>>> for row in df.itertuples():  
...     print(row)  
...  
Pandas(Index='dog', num_legs=4, num_wings=0)  
Pandas(Index='hawk', num_legs=2, num_wings=2)
```



Pandas : Difference

```
>>> df = pd.DataFrame({'a': [1, 2, 3, 4, 5, 6],  
...                    'b': [1, 1, 2, 3, 5, 8],  
...                    'c': [1, 4, 9, 16, 25, 36]})  
>>> df
```

	a	b	c
0	1	1	1
1	2	1	4
2	3	2	9
3	4	3	16
4	5	5	25
5	6	8	36

```
>>> df.diff()
```

	a	b	c
0	NaN	NaN	NaN
1	1.0	0.0	3.0
2	1.0	1.0	5.0
3	1.0	1.0	7.0
4	1.0	2.0	9.0
5	1.0	3.0	11.0



Pandas : Maximum / Minimum

```
>>> idx = pd.MultiIndex.from_arrays([
...     ['warm', 'warm', 'cold', 'cold'],
...     ['dog', 'falcon', 'fish', 'spider']],
...     names=['blooded', 'animal'])
>>> s = pd.Series([4, 2, 0, 8], name='legs', index=idx)
>>> s
blooded  animal
warm     dog      4
         falcon   2
cold     fish     0
         spider   8
Name: legs, dtype: int64
```

```
>>> s.max()
8
```

Pandas : Equals

```
>>> df = pd.DataFrame({'1': [10], '2': [20]})
>>> df
   1  2
0 10 20
```

```
>>> exactly_equal = pd.DataFrame({'1': [10], '2': [20]})
>>> exactly_equal
   1  2
0 10 20
>>> df.equals(exactly_equal)
True
```

```
>>> different_column_type = pd.DataFrame({'1.0': [10], '2.0': [20]})
>>> different_column_type
   1.0  2.0
0   10   20
>>> df.equals(different_column_type)
True
```



Pandas : Equals

```
>>> df = pd.DataFrame({'1': [10], '2': [20]})
>>> df
   1  2
0 10 20
```

```
>>> different_data_type = pd.DataFrame({'1': [10.0], '2': [20.0]})
>>> different_data_type
   1    2
0 10.0 20.0
>>> df.equals(different_data_type)
False
```



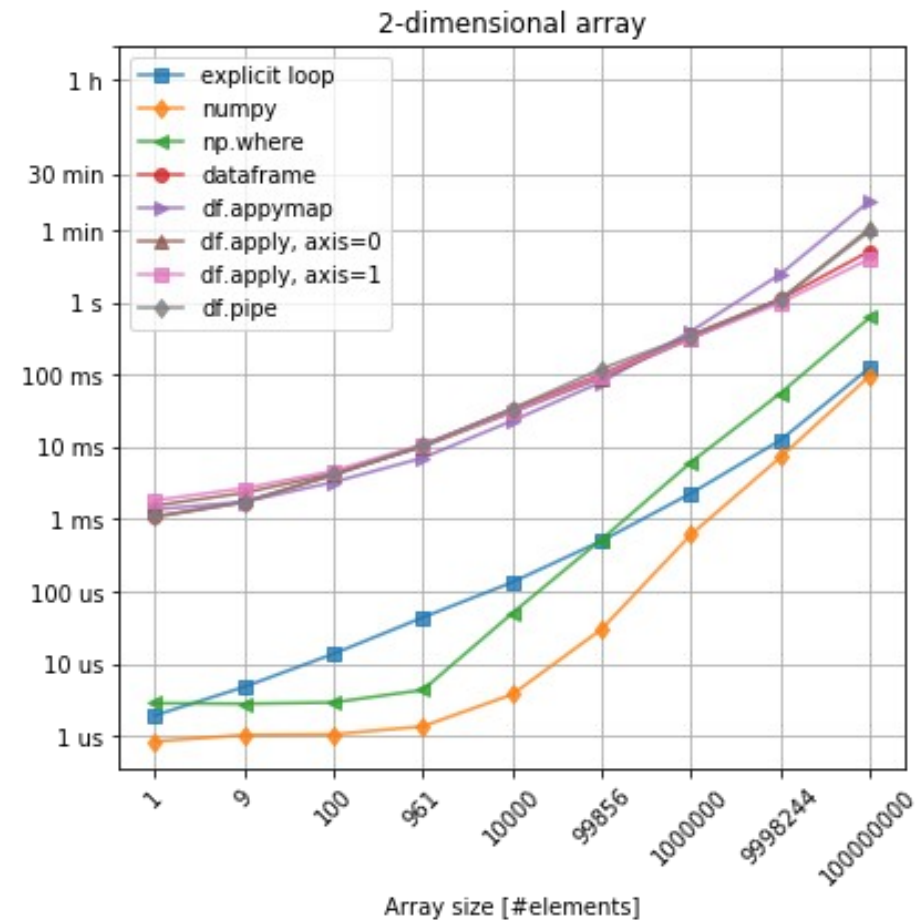
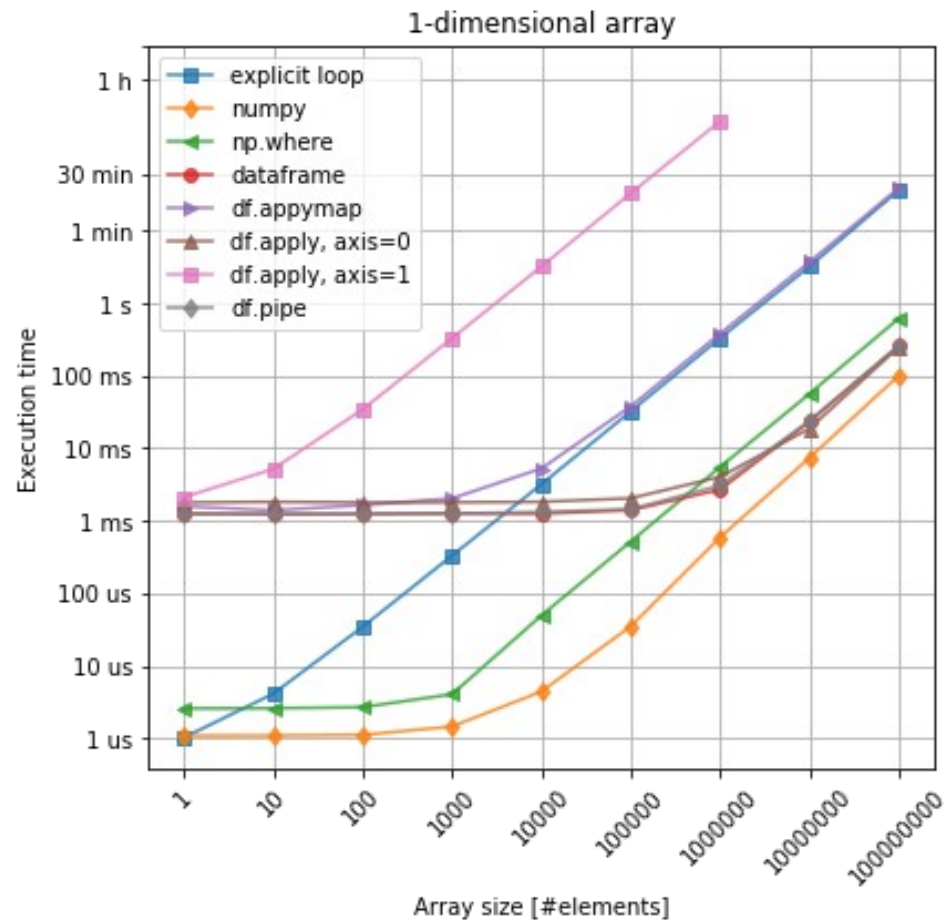
Pandas : Renaming

```
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
>>> df.rename(columns={"A": "a", "B": "c"})
```

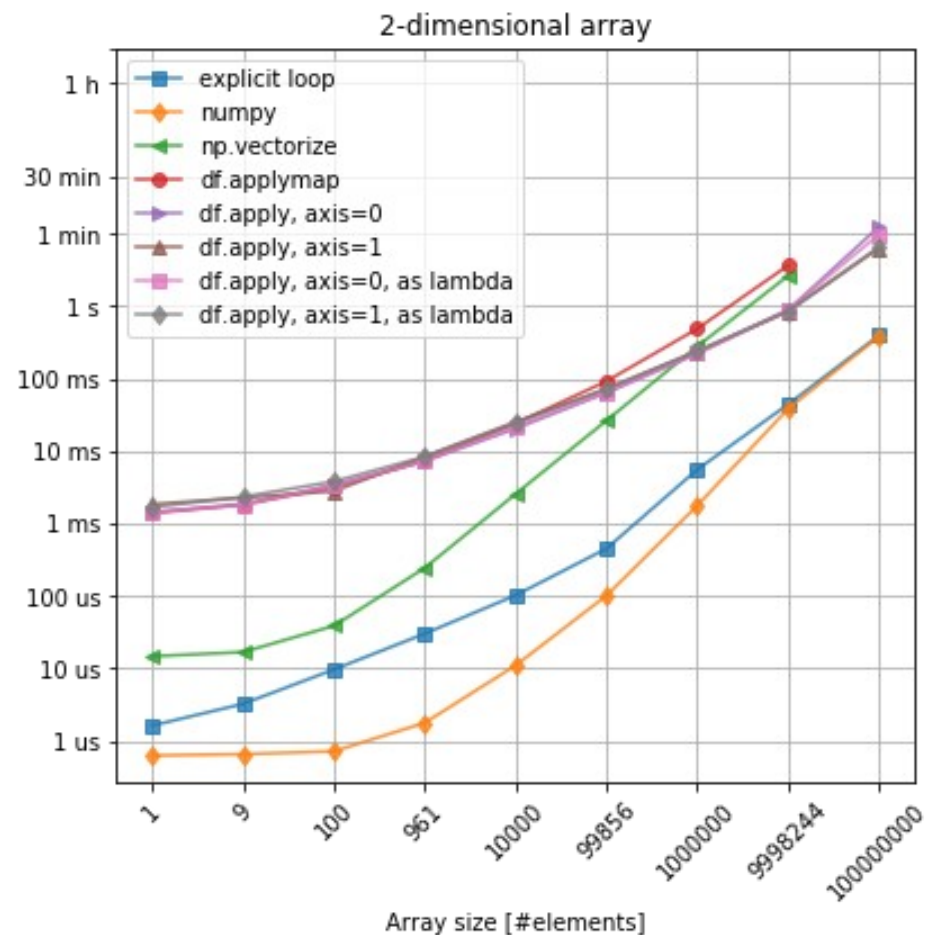
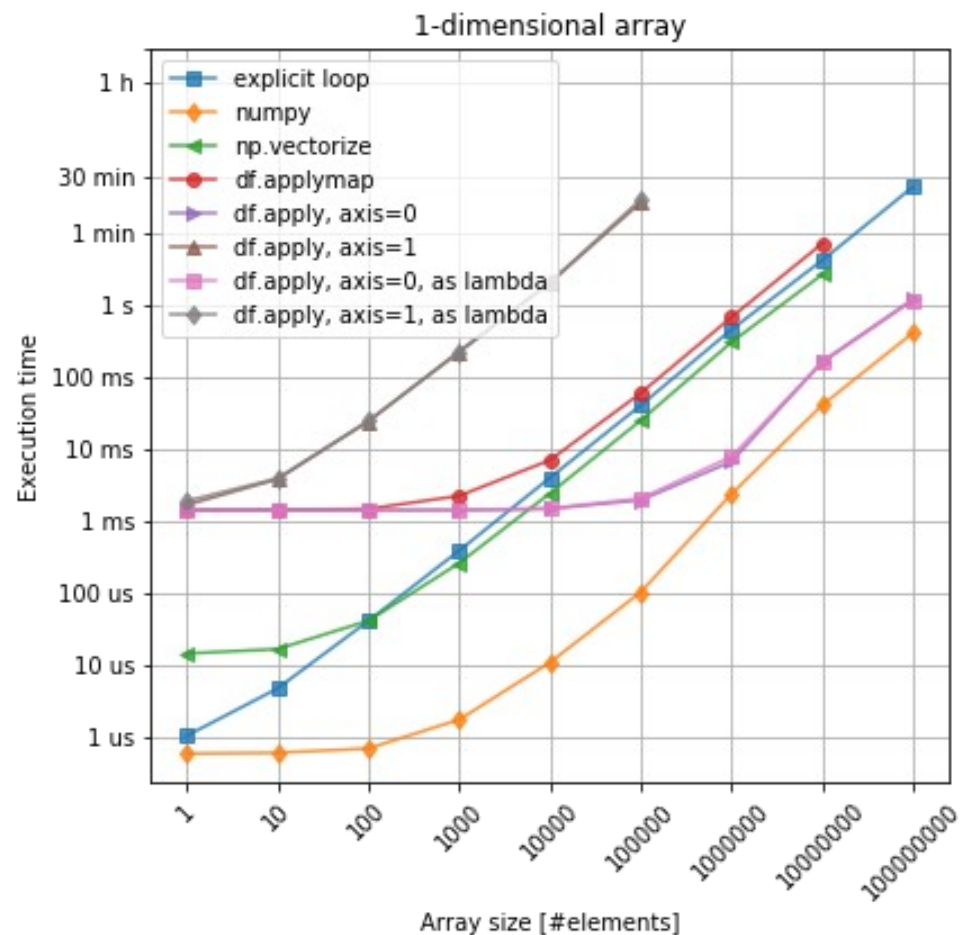
	a	c
0	1	4
1	2	5
2	3	6



Pandas : Performances selection



Pandas : Performances map



Pandas : Performances summary

- Use apply, applymap and pipe when dataframes entries $> 10^6$
- Try to make your heavy computation outside a Pandas DataFrame



Pandas : Cross use

Pandas DataFrame allows you to easily use such libraries :

